

```

    # See if user clicked
    ❸ if event.type == pygame.MOUSEBUTTONDOWN:
        # mouseX, mouseY = event.pos # Could do this if we needed it

        # Check if the click was in the rect of the ball
        # If so, choose a random new location
        ❹ if ballRect.collidepoint(event.pos):
            ballX = random.randrange(MAX_WIDTH)
            ballY = random.randrange(MAX_HEIGHT)
            ballRect = pygame.Rect(ballX, ballY, BALL_WIDTH_HEIGHT,
                                   BALL_WIDTH_HEIGHT)

# 8 Do any "per frame" actions

# 9 - Clear the window
window.fill(BLACK)

# 10 - Draw all window elements
# Draw the ball at the randomized location
7 window.blit(ballImage, (ballX, ballY))

# 11 - Update the window
pygame.display.update()

# 12 - Slow things down a bit
clock.tick(FRAMES_PER_SECOND) # make pygame wait

```

Listing 5-3: Detecting a mouse click and acting on it

Since we need to generate random numbers for the ball coordinates, we import the `random` package ❶.

We then add a new constant to define the height and width of our image as 100 pixels ❷. We also create two more constants to limit the maximum width and height coordinates. By using these constants rather than the size of the window, we ensure that our ball image will always appear fully within the window (remember that when we refer to the location of an image, we are specifying the position of its upper-left corner). We use those constants to choose random values for the starting x- and y-coordinates for our ball ❸.